Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский технологический университет «МИСИС»

Институт компьютерных наук (ИКН)

Кафедра Инфокоммуникационных технологий (ИКТ)

Лабораторная работа №2

по дисциплине «Технологии программирования»

на тему «Основы ООП: реализация иерархии классов»

Выполнила:

студентка группы БИВТ-25-4-5

Шепилова Дарья Михайловна

Проверил:

Степан Пискунов

Москва, 2026

Цель работы: изучить основы объектно-ориентированного программирования (ООП) и на практике реализовать иерархию классов для пошаговой стратегии на языке C#: использовать наследование, абстрактные классы, интерфейсы, инкапсуляцию и полиморфизм.

Ход выполнения работы:

1. Постановка задачи и структура классов

Нужно реализовать иерархию классов для игры-стратегии. Работа выполнена на C# в Visual Studio. Был создан проект типа Console App с именем LABB2.

Структура:

• GameObject – базовый абстрактный класс всех игровых объектов

• Unit (абстрактный) и Building – наследники GameObject

• Archer – наследник Unit, реализует интерфейсы Attacker и Moveable

• Fort – наследник Building, реализует Attacker

• MobileHome– наследник Building, реализует Moveable

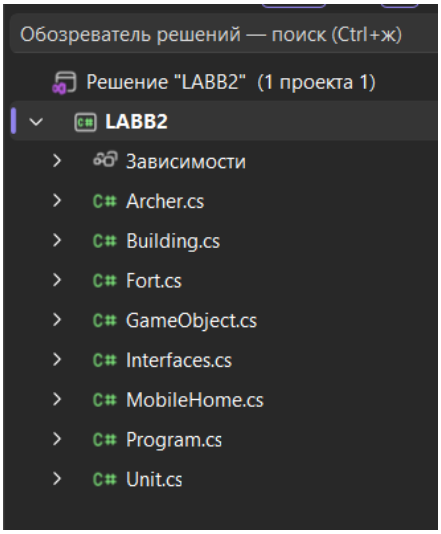


Рис. 1. Структура проекта.

2. Класс GameObject

Абстрактный базовый класс GameObject:

Поля Id, Name — private readonly (нельзя менять после создания). Координаты X, Y — protected (доступны наследникам для MoveTo). Статический счётчик count — уникальные id для всех объектов. Методы GetId, GetName, GetX, GetY — геттеры из задания. Для удобного вывода информации об объекте был переопределён ToString(). Снаружи new GameObject(...) невозможен из-за protected.

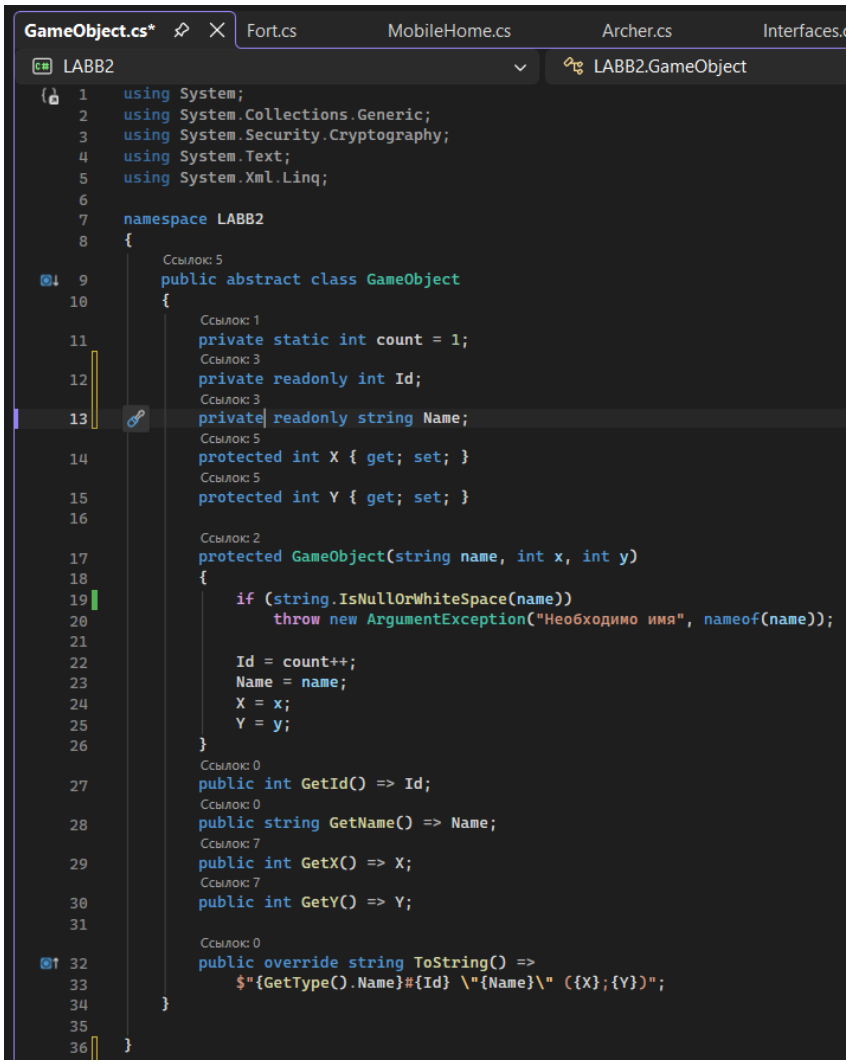


Рис. 2. Класс GameObject.

3. Класс Unit

Класс Unit наследуется от GameObject и описывает юнита:

Поля Hp и maxHp (private, maxHp — readonly). Конструктор вызывает base(name, x, y) для инициализации родителя с проверкой, что maxHp > 0. Метод IsAlive() — возвращает true, если Hp > 0. Метод GetHp() — текущее состояние. Метод ReceiveDamage(float damage) — метка virtual, чтобы наследники могли переопределить. Урон не может быть отрицательным, HP не становится ниже нуля.

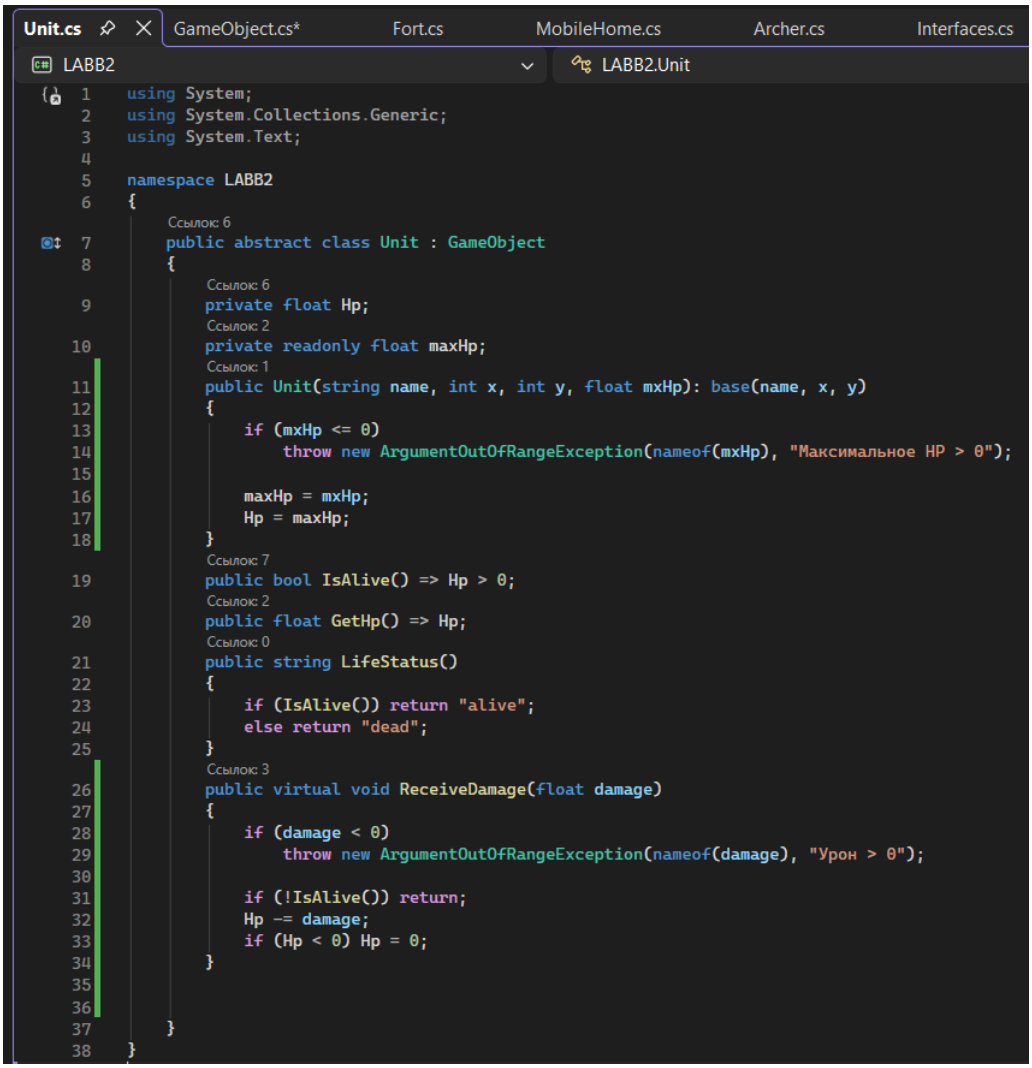


Рис. 3. Класс Unit.

4. Класс Building

Класс Building наследуется от GameObject и описывает постройку:

Поля b (готова ли), buildtime (за сколько ходов строится), turns (сколько ходов уже потрачено). Конструктор принимает bool b = false — параметр по умолчанию. IsBuilt() — возвращает true, если построена. OnTurn() — метка virtual, вызывается раз в ход и увеличивает turns. Когда счётчик достигает buildtime — постройка готова.

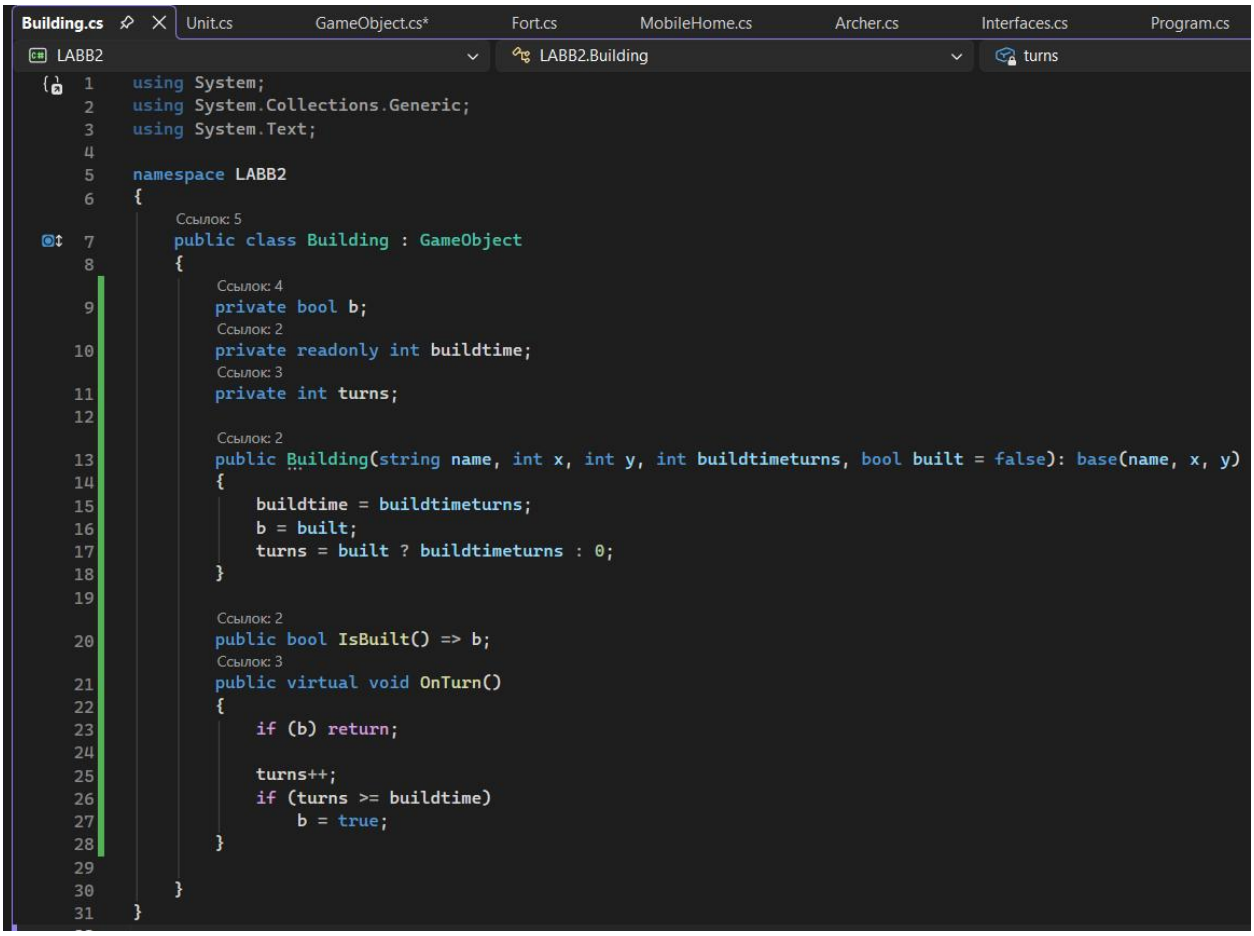


Рис. 4. Класс Building.

5. Интерфейсы Attacker и Moveable

Были созданы два интерфейса:

Attacker с методом void Attack(Unit unit) — контракт «умею атаковать юнита».

Moveable с методом void MoveTo(int x, int y) — контракт «умею двигаться по смещению». Они нужны, чтобы объединить классы с разной природой (юниты и постройки) по общим возможностям: например, атаковать могут и лучник, и крепость.

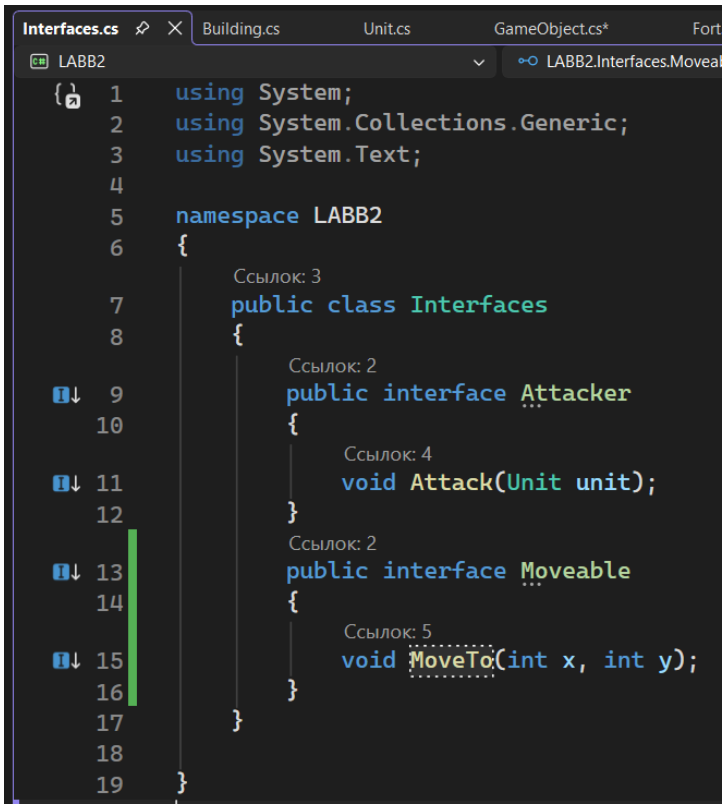


Рис. 5. Интерфейсы Attacker и Moveable.

6. Класс Archer

Класс Archer наследует Unit и реализует оба интерфейса — Attacker и Moveable:

Конструктор вызывает base(name, x, y, maxHp) и сохраняет урон и дальность. Attack(Unit unit) — проверяет, что цель не null, что сам жив, что цель жива, и

что расстояние до точки (сумма разниц координат по модулю) не больше дальности. MoveTo(int x, int y) — если жив, меняет X и Y на смещение.

Archer — одновременно юнит (наследует Unit) и умеет атаковать/двигаться (реализует интерфейсы).

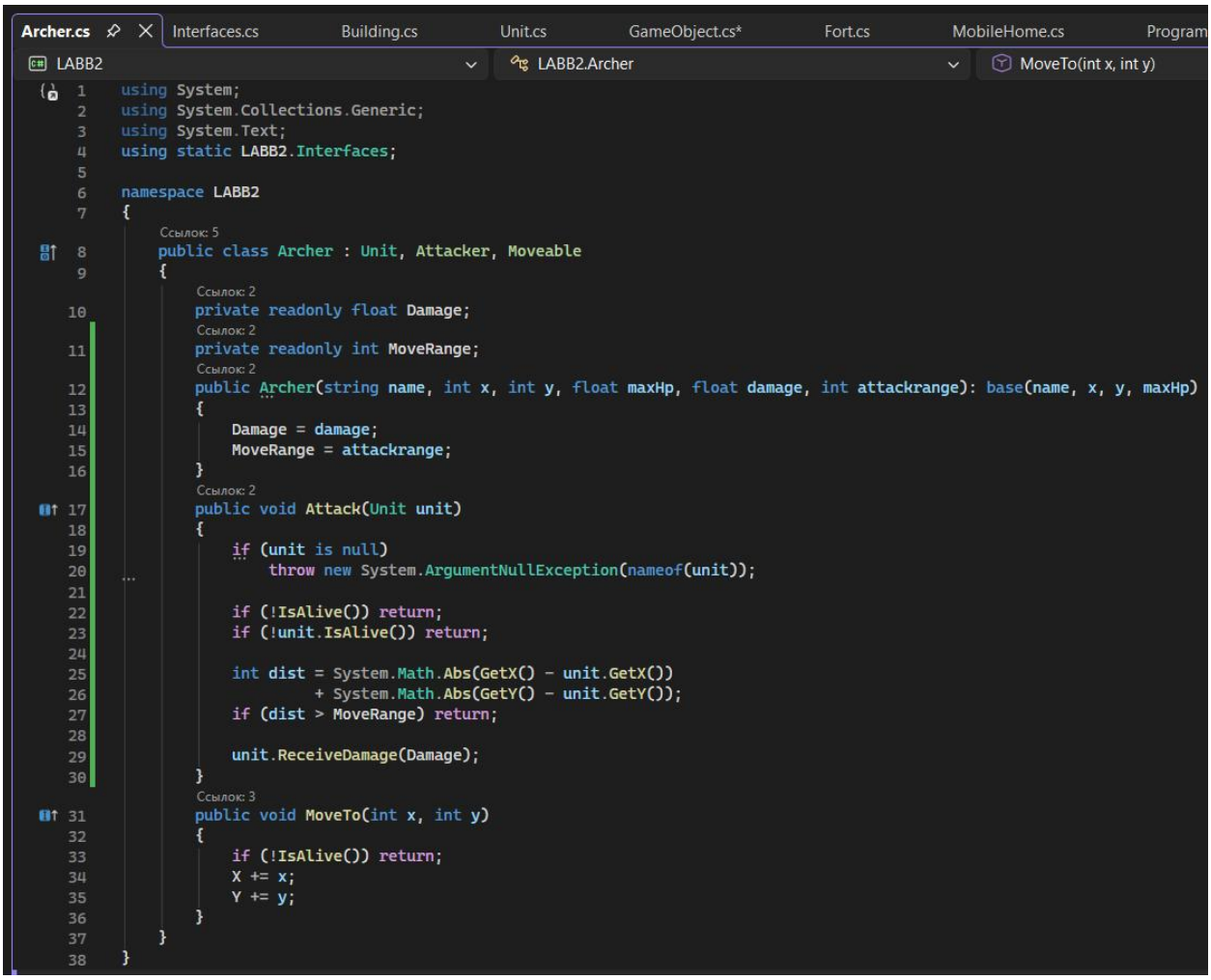


Рис. 6. Класс Archer.

7. Класс Fort

Класс Fort наследует Building и реализует Attacker:

Конструктор вызывает base(name, x, y, buildtimeturns, built) и сохраняет урон и дальность. Attack(Unit unit) — проверяет null, что крепость построена

(IsBuilt()), что цель жива, и что расстояние не больше дальности и далее наносит урон. Недостроенная крепость не стреляет, потому что атака защищена проверкой if (!IsBuilt()) return;.

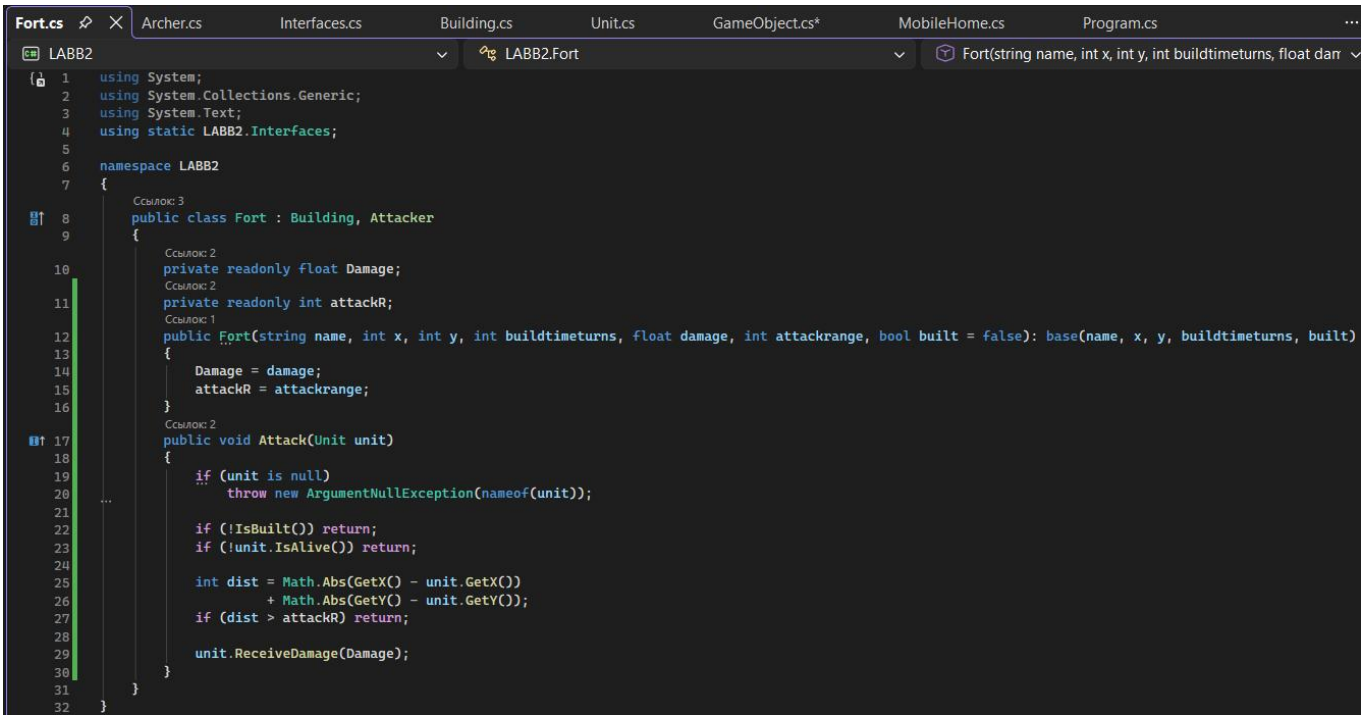


Рис. 7. Класс Fort.

8. Класс MobileHome

Класс MobileHome наследует Building и реализует Moveable:

Конструктор просто передаёт параметры в Building. MoveTo(int x, int y) — если дом построен (IsBuilt()), меняет координаты. Недостроенный дом не едет. И Archer, и MobileHome реализуют Moveable, но с разными проверками: лучник — по IsAlive(), дом — по IsBuilt().

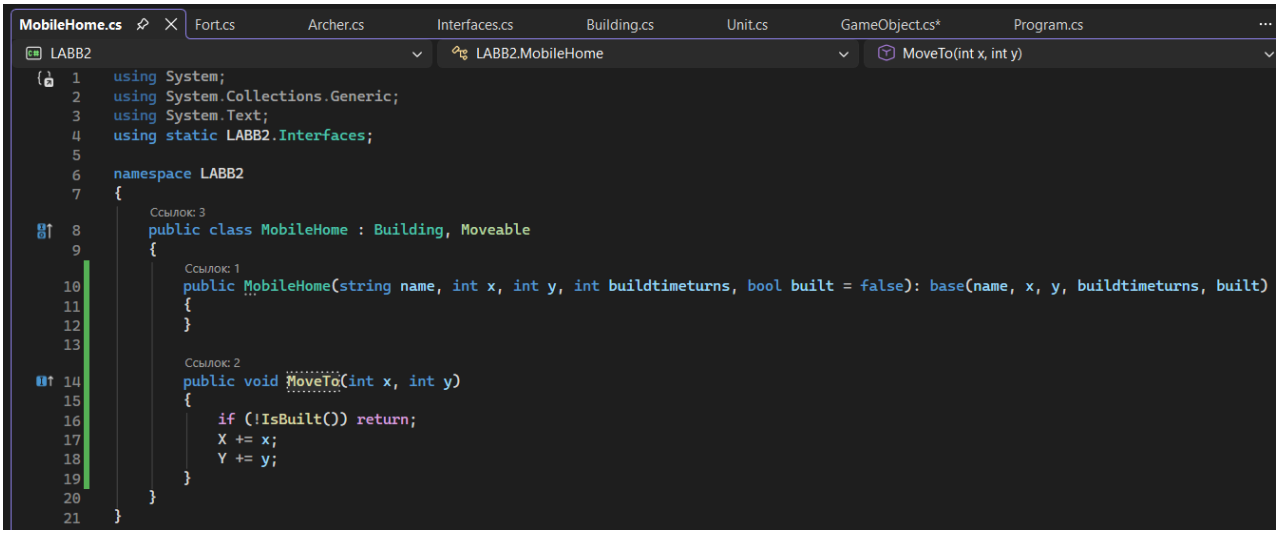


Рис. 8. Класс MobileHome.

9. Проверка работы в классе Program

В Main() были созданы 4 объекта (2 лучника, дом, крепость):

Печать начального состояния через ToString().

fort.OnTurn() x 2 — крепость строится

cart.OnTurn() + cart.MoveTo(10, 10) — дом строится и едет.

lenya.Attack(kostya) — лучник атакует лучника.

lenya.MoveTo(3, 0) — лучник двигается.

fort.Attack(lenya) — крепость стреляет.



Рис. 9. Класс Program.

10. Результат работы программы

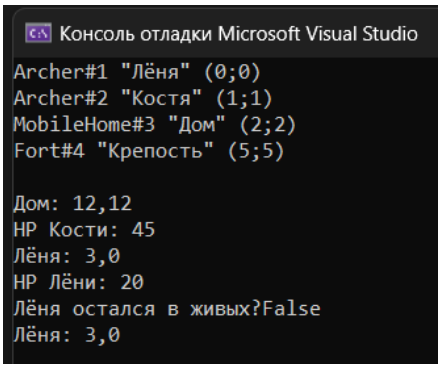


Рис. 10. Окно консоли с выводом.

Вывод:

В ходе работы была реализована иерархия классов для пошаговой стратегии на C#. Базовый класс GameObject и абстрактные Unit и Building хранят общие данные и поведение, а конкретные классы Archer, Fort и MobileHouse дополняют их своими возможностями через интерфейсы Attacker и Moveable. На практике были применены основные принципы ООП: инкапсуляция (защищены поля Id, Name, Hp, maxHp т.к объявлены private, координаты X, Y — protected, а доступ снаружи идёт только через методы), наследование (общие поля и методы не дублируются), полиморфизм (переопределение ToString() и виртуальный ReceiveDamage()) и абстракция (абстрактные классы и интерфейсы). Тестовый запуск показал, что все классы работают корректно, включая проверки состояния.